

YI-LEI LI

Newark, DE |  +1 302-610-1010 |  ray1998514@gmail.com |  LinkedIn |  GitHub

Summary

Software engineer with an **M.S. in Computer Science** (University of Delaware, May 2026) who ships tested, production-style systems: a **Java 21 / Spring Boot 3** banking service with atomic transfers and CI, a **TypeScript ETL pipeline** processing 5,000+ records per run, and a **Python backtesting engine** built around walk-forward out-of-sample validation. Prior backend engineering internship at Cathay Financial Holdings. Also strong on the data side — SQL, scikit-learn/XGBoost, and LLM integration.

Technical Skills

Languages:	Java, TypeScript/JavaScript (Node.js), Python, SQL
Backend:	Spring Boot 3, JPA/Hibernate, REST API design, transactions & concurrency control (pessimistic locking), Spring Boot Actuator
Frontend:	React, Vite, Tailwind CSS, HTML5/CSS3
Databases:	PostgreSQL (Row-Level Security), H2, DuckDB; window functions, CTEs
Testing & DevOps:	JUnit 5, MockMvc, Vitest, test automation, browser automation (Playwright), Maven, CI/CD (GitHub Actions), Git/GitHub
ML & AI:	scikit-learn, XGBoost, SHAP, LLM integration (Anthropic Claude API), AI-assisted development

Projects

Banking Transactions API — Java 21, Spring Boot 3, PostgreSQL [\[GitHub\]](#)

- RESTful core-banking service: accounts, deposits/withdrawals with overdraft rejection, and **atomic transfers** — both legs commit or neither does (@Transactional rollback).
- Prevented double-spend and deadlock under concurrency with **ordered pessimistic row locks** (SELECT ... FOR UPDATE, ascending-id acquisition order); every movement writes an **immutable audit-ledger** row with post-transaction balance.
- **19 JUnit/MockMvc tests** (service rules + HTTP error contracts: 201/400/404/409) run in **GitHub Actions CI** on every push; **Actuator** health/metrics endpoints with a real DB connectivity check.

Job-Opening Monitor — Node.js, TypeScript, Supabase [\[GitHub\]](#)

- Designed a scheduled **ETL pipeline** processing ~5,000+ postings per run across three heterogeneous ATS APIs (Workday CXS, Greenhouse, Lever) — one adapter per platform behind a single normalization layer, with **bounded-concurrency** fetches and zero runtime dependencies.
- Made re-runs **idempotent** via a Postgres state table (never double-alerts); scheduled on a GitHub Actions cron with Telegram + email alerts; **32 Vitest unit tests** over matching and classification logic.

quantlab — Multi-Market Backtesting & Research Engine — Python, pandas, NumPy [\[GitHub\]](#)

- Vectorized backtester for equities, crypto, and prediction markets with **next-bar execution** (eliminating look-ahead bias), fee/slippage modeling, and buy-and-hold benchmarking; **8 strategies** behind one interface with registry auto-discovery, so a new strategy works with the optimizer and validator without engine changes.
- Built **anchored walk-forward validation** reporting **in-sample vs out-of-sample Sharpe per fold** to expose overfit parameters — a documented case study reaches **1.29 OOS Sharpe** at -17.7% drawdown, with an appendix recording three of four higher-return attempts *failing* out-of-sample.

Credit Card Default Risk Analysis — Python, SQL, XGBoost/SHAP [\[GitHub\]](#)

- Portfolio risk analytics on **30,000 customers**: five SQL analyses (window functions, CTEs), logistic vs **XGBoost** models (held-out AUC **0.777**) with **SHAP**; top decile captures 31% of defaults (3.1x lift). Includes a grounded **LLM** layer that drafts the weekly risk memo.

Education

University of Delaware <i>Master of Science in Computer Science — GPA 3.66/4.0</i>	Newark, DE May 2026
National Kaohsiung University of Science and Technology <i>Bachelor of Science in Electrical Engineering</i>	Kaohsiung, Taiwan 2020

Experience

Cathay Financial Holdings Co., Ltd. Jul 2023 – Nov 2023
Back-End Software Engineering Intern Taipei, Taiwan

- Delivered a CRUD service (**Java + SQL**) as part of an **Agile** development team, and traced the firm's transaction-processing flow end to end; studied how the system enforces correctness — sufficient-balance checks before transfers and concurrency control serializing transactions per account (**ACID** integrity and isolation).

Graduate Research Assistant — Chang Gung University 2021 – 2022
Researched **Neural Radiance Fields (NeRF)** for 3D scene reconstruction from multi-view images; earlier built a full-stack IoT sensor-monitoring dashboard (REST + WebSocket) as an undergraduate research assistant.